Fashion Category Tagging

MAT4091: Basic Deep Learning and Applications

Sehyun Kwon | 2026 Fall IC-PBL Project | In collaboration with Dalpha

1 Task

Build a model that takes a fashion image and returns every target clothing or accessory category visible in it. This is a *multi-label* task: one image may have several correct categories. Predict the union of the categories visible across all people in the image. If none of the 27 target categories is present, return an empty set. The required output is an image-level tag set, not bounding boxes or segmentation masks.

The 27 categories, in the order used by the provided label files, are: *shirt, blouse*; *top, t-shirt, sweatshirt*; *sweater*; *cardigan*; *jacket*; *vest*; *pants*; *shorts*; *skirt*; *coat*; *dress*; *jumpsuit*; *cape*; *glasses*; *hat*; *headband, head covering, hair accessory*; *tie*; *glove*; *watch*; *belt*; *leg warmer*; *tights, stockings*; *sock*; *shoe*; *bag, wallet*; *scarf*; and *umbrella*. Categories with commas in their names are single categories. For example, *pants* and *shorts* are separate outputs.

2 Data and examples

The LMS will provide labeled training and validation ZIP files. It will also provide a small *unlabeled* public test sample, so you can inspect the expected input format and explore how your model behaves on new images. The final evaluation set and its labels will remain hidden. Do not assume that the validation images fully represent the final evaluation images.

Figure 1 shows three actual images from the provided Fashionpedia training split. The category sets below the images are the corresponding training labels, restricted to the 27 target categories.



Labels: shirt, blouse; jacket; pants



Labels: dress; cape; shoe



Labels: dress; shoe; umbrella

Figure 1: Example input images and their image-level target categories. These examples are from the training split, not the test set.

3 Rules and constraints

- The total number of parameters used to produce a final prediction must not exceed **50 million**. For an ensemble, count the parameters of every inference model.
- You may start from publicly available pretrained weights. You may also collect or generate additional *training* images, provided that you document their sources, labels, and use. Do not train on, manually label for training, or tune thresholds against the public test sample or any final test image.
- AI tools are permitted for programming, experiment design, and analysis. The final predictions must come from the submitted, locally runnable model and code, not an external inference API.
- Choose architectures, checkpoints, post-processing, and thresholds using the training and validation data. The unlabeled public test sample is for inspection, not for supervised model selection.
- Your submission must be reproducible: include the model weights, inference code, dependencies, and a command that processes a folder of images into predictions. Preserve the exact category names above; return an empty list when no category is predicted.

4 Evaluation

We will report **Micro F1**, **Macro F1**, and **Image-wise F1** on the hidden evaluation set. Micro F1 pools decisions across all images and categories; Macro F1 averages the 27 per-category F1 scores; Image-wise F1 averages the score of each image's predicted tag set. A prediction and ground truth that are both empty receive an Image-wise F1 of 1. We will also inspect errors on images for which no target category is present. The final evaluation protocol and any score weighting will be announced before the submission deadline.

5 Deliverables and presentation

Submit the following through the LMS: (1) executable training and inference code; (2) the final checkpoint; (3) a short README with exact reproduction commands and the parameter count; and (4) a final paper in the **CVPR template**. The paper must contain **six pages of main text**; references do not count toward the six pages. The CVPR template will be uploaded separately. In the paper, describe your baseline, improvements, validation results, failure analysis, any extra data, and what you learned from the experiments. Do not place required content in an appendix to bypass the page limit.

Each group will also give a project presentation at the end of the semester. Present the model, the evidence behind your design choices, important successes and failures, and the contributions of the group members. Submission dates and presentation logistics will be posted on the LMS.

6 Model submission format

Submit one self-contained ZIP file per group. The submitted system must run on a folder of images without Internet access and produce the final predicted category names.

Required files

```
team_name.zip
|-- predict.py
|-- model.pt
|-- requirements.txt
`-- README.md
```

The README must state the exact execution command and the total parameter count. Include every weight and dependency needed for offline inference.

Required command

```
python predict.py --input_dir TEST_IMAGES --output predictions.jsonl
```

Required final output

The final output must contain **predicted class names**, not logits, probabilities, or class indices. Use exactly the names in `categories.txt`.

```
{"test_id":"test5k_000001","labels":["jacket","pants","shoe"]}
{"test_id":"test5k_000002","labels":[]}
```

Output rules

- Write one JSON object per image.
- Use the image filename without its extension as `test_id`.
- Multiple predicted categories must be returned as a list of class names.
- Return an empty list, `[]`, when no target category is predicted.
- Do not return unknown names or repeat the same class name.
- The order of names inside the list does not affect evaluation.

Model-side post-processing

Preprocessing, score conversion, threshold selection, calibration, and the index-to-name mapping must all be included in the submitted inference system. Any thresholds must be selected using only the training and validation data and must be fixed before private evaluation.

Before submission

Run the complete command on the public test images and confirm that every image produces one valid record. Submissions that cannot be executed or do not follow this output format may not be graded correctly.